The Spaghetti Engine
VISUAL SYSTEM MODELLING WORKSPACE

The Spaghetti Engine

Methodological Annex

The Formal, Mathematical, and Theoretical Basis of Every Computational Method Implemented in the Tool

Alfred Paarlberg

To be read alongside: The Spaghetti Engine - User Manual (version 0.5.3)

Contents

1. Purpose and Scope of This Annex 4

2. Theoretical Foundations 4

2.1 From cognitive maps to fuzzy cognitive maps 4

2.2 Why FCMs suit participatory policy modelling 5

2.3 FCMs for nexus analysis: modules, interfaces, and scales 5

3. Formal Model Specification 5

3.1 State representation 5

3.2 The update rule 6

3.3 Squashing functions 6

3.4 Convergence and equilibrium 7

3.5 Synchronous vs. asynchronous updating 8

3.6 Initial conditions and multiple equilibria 8

4. Modules and the Analysis Scope 8

4.1 Module assignment 9

4.2 The analysed network 9

4.3 Methodological consequences 9

4.4 Traceability 10

4.5 The folded (sub-model) display 10

4.6 Integrity checks on opening a model 10

4.7 Automatic layout of the map 11

4.8 Figure export 13

5. Structural (Graph-Theoretic) Analysis 13

5.1 Degree-based centrality 13

5.2 Driver and receiver scores 14

5.3 Role classification 14

5.4 Betweenness centrality 14

5.5 Density, connectivity, and centralization 15

5.6 Clustering coefficient 15

5.7 Modularity of the module partition 15

5.8 Interface relationships 16

5.9 Structural complexity index 16

5.10 Feedback loop detection and classification 16

6. Perturbation-Based Influence and Sensitivity Scores 16

6.1 Rationale 16

6.2 Formal procedure 16

6.3 A known limitation of this design 17

7. Multi-Scale Analysis 17

7.1 Scales 17

7.2 Ranking 18

7.3 "Importance mainly from" 18

7.4 Leverage points by scale 18

7.5 Limitations 18

8. Influence Routes and Feedback Loops 19

8.1 Routes and their enumeration 19

8.2 Strength and sign 19

8.3 Route types 19
8.4 Bridge concepts, boundary crossings, and bottlenecks 20
8.5 Transmission between modules 20
8.6 Feedback loops by module 20
8.7 Limitations 20

9. Scenario Analysis Methodology 21
9.1 The driver/lock formalism 21
9.2 Effect size, synergy, and trade-off 21

10. Sensitivity Analysis (Dedicated Tab) 21
10.1 Elasticity 21
10.2 Tornado ranking 22

11. Monte Carlo Simulation 22
11.1 Sampling design 22
11.2 Summary statistics 22
11.3 Randomness and reproducibility 23
11.4 Interpretation and limitations 23

12. Transition Point Analysis 24
12.1 Procedure 24
12.2 Response-type classification 25
12.3 Efficiency 25

13. Transition Pathway Analysis 26
13.1 Outcomes, desirability, and outcome gain 26
13.2 Levers and the equilibrium operator 26
13.3 Stages and conditional transition points 26
13.4 Lever profiles and the candidate pool 27
13.5 Candidate construction 27
13.6 Pathway metrics 28
13.7 Admissibility, archetypes, and the efficient frontier 28
13.8 Implementation portfolios and prefix regret 29
13.9 Invariance test for path dependence 29
13.10 Simulation settings and convergence 30
13.11 Robustness to simulation settings 30
13.12 Assumptions and limitations 30

14. Validity, Assumptions, and Limitations 31
14.1 Assumptions inherited from the FCM formalism itself 31
14.2 Assumptions introduced by modules and scope 31
14.3 Method-specific limitations 31
14.4 What this tool does not claim 32
14.5 The former example models 32

15. Summary Table 34

16. References 35

1. Purpose and Scope of This Annex

The Spaghetti Engine - User Manual documents how to use each feature of the tool. This annex documents the mathematical and theoretical basis for why each feature works the way it does, so that a number reported from the tool (an equilibrium, an influence score, a betweenness value, a route strength, a transition point) can be traced back to a defensible method, not treated as a black box. It describes version 0.4.0 of the tool. Compared with version 0.3.0, this version adds one analytical method, Monte Carlo simulation (Section 11), which propagates the uncertainty in the relationship weights through the model; and one simulation option, the linear update without a squashing function (Sections 3.3 and 3.4). The methods of the earlier versions are otherwise unchanged; version 0.3.0 added Transition Pathway Analysis (Section 13), which combines intervention levers into ordered pathways, each stage implemented up to its transition point in the state reached by the preceding stages; version 0.2.1 added the folded (sub-model) display of modules (Section 4.5), integrity checks on imported and restored models (Section 4.6), and the bundled example model (Section 14.5).

Every method described here falls into one of two categories, and this annex is explicit about which is which throughout:

- Established methods drawn from the Fuzzy Cognitive Mapping, graph-theory, network-science, and sensitivity-analysis literature, with a citation to the specific source.
- Original implementation choices made for this tool (an operational threshold, a classification rule, an analysis not previously published in this exact form), presented as such, with the reasoning behind the choice and its known limitations stated plainly.

> **On this distinction:** Conflating the two would misrepresent the tool's methodological standing. Where a threshold or rule below has no citation, that is deliberate: it is this tool's own design decision, not an omitted reference.

Section 2 sets out the theoretical lineage the tool sits in. Section 3 gives the formal model specification, including the option of running without a squashing function, and Section 4 formalizes modules and the analysis scope, which determine the network every other method operates on, together with the folded display, the integrity checks applied when a model is opened, and the automatic layout of the map. Sections 5 to 13 cover each analytical method: structural analysis (5), perturbation-based scores (6), multi-scale analysis (7), influence routes and feedback loops (8), scenarios (9), sensitivity analysis (10), Monte Carlo simulation (11), transition point analysis (12), and transition pathway analysis (13). Section 14 states the assumptions and limitations that follow from these methods: what the tool's output can and cannot be taken to mean. Section 15 is a summary table, and Section 16 lists full references.

2. Theoretical Foundations

2.1 From cognitive maps to fuzzy cognitive maps

Cognitive mapping, representing an actor's or group's beliefs about a system as a signed directed graph of causal concepts, originates with Axelrod (1976), who used cognitive maps to formalize the causal reasoning of political elites, with each relationship signed simply positive, negative, or absent (Axelrod, 1976).

Kosko (1986) extended this in two ways that make the maps computable rather than only graph-theoretically traceable. First, he replaced Axelrod's discrete (+/-/0) causal signs with continuous, "fuzzy" weights: a relationship can be weakly or strongly positive or negative, not only present or absent. Second, he formalized the map's behavior as a discrete-time dynamical system: concept activations are updated iteratively from their neighbors' activations, in the same general form as a recurrent (Hopfield-style) neural network, so a map's implications can be simulated numerically to a settled state rather than reasoned about only qualitatively (Kosko, 1986). This numerically simulable formalism, fuzzy cognitive maps (FCMs), is the formalism The Spaghetti Engine implements directly: its "concepts" and "relationships" are Kosko's concept nodes and fuzzy causal edges, and every simulation-based analysis in the tool operates on the same update rule described in Section 3.

2.2 Why FCMs suit participatory policy modelling

FCMs let stakeholders express what they believe drives a system (which factors matter, and whether each relationship helps or hinders) without requiring precise, measured quantitative parameters, which are frequently unavailable for the kind of complex, multi-stakeholder policy systems this research addresses. A group can co-construct a map by agreeing on concepts and roughly how strongly and in which direction one influences another, and the resulting structure remains formal enough to be simulated, compared across scenarios, and interrogated with graph-theoretic and sensitivity methods. This combination of qualitative elicitation and quantitative simulation is the methodological rationale for using an FCM tool for this research, rather than either a purely qualitative causal map or a fully specified quantitative system-dynamics model.

2.3 FCMs for nexus analysis: modules, interfaces, and scales

A nexus is a system of several interacting subsystems, for example water, energy, food, ecosystems, and the policy domains that govern them. From version 0.2, the tool supports this directly. Each concept can be assigned to a module (a subsystem), which divides the map's relationships into two kinds: relationships within a module, which describe how a subsystem works internally, and relationships between modules, which describe how the subsystems are coupled (the nexus interface). Every analysis can then be applied at several scales: the whole system, one subsystem in isolation, a combination of subsystems, or the interface between them.

This framing is the tool's own operationalization and is deliberately generic: modules are whatever categories the modeller assigns, and nothing about any particular nexus is built into the methods. Its analytical value lies in comparison. The same concept can be central within its subsystem but peripheral in the system as a whole, or unremarkable inside any subsystem but decisive as a bridge between them; distinguishing these cases is the purpose of the scope mechanism (Section 4), the multi-scale analysis (Section 7), and the route and loop analysis (Section 8).

3. Formal Model Specification

3.1 State representation

A model is a set of n concepts $C = \{c1, \ldots, cn\}$, each with an activation $ci(t) \in [-1, +1]$ at discrete time step t, collected into the activation vector A(t). Relationships are represented by a weight matrix W, where $wij \in [-1, +1]$ is the weight of the directed relationship from ci to cj ($wij = 0$ where no relationship is drawn). A(0) is the initial activation vector: the model's starting state before any iteration. Each concept may also carry a category label, which defines its module (Section 4).

3.2 The update rule

At each iteration, every concept's new activation is computed from the current activations of the concepts with an incoming relationship to it, passed through a bounded squashing function f (Section 3.3) or, if squashing is switched off, left as it is. Two variants are implemented:

> Relative (default): $A(t+1) = f(\, A(t) + W \cdot A(t) \,)$

> Absolute: $A(t+1) = f(\, W \cdot A(t) \,)$

The Absolute form is Kosko's (1986) original FCM update rule: a concept's new value comes purely from incoming influence, and its own previous value is discarded at every step. The Relative form is a widely used variant in later FCM tools (including this one, by default) in which a concept's new value builds on top of where it currently is, rather than being fully replaced each iteration; this produces a smoother, more inertial trajectory toward equilibrium. Both are offered so results can be checked against either convention.

3.3 Squashing functions

The squashing function f keeps every activation within [-1, +1] regardless of how large the summed incoming influence is. Three are implemented, and the identity is offered as a fourth option (below):

Function	Definition	Properties
Hyperbolic tangent (default)	f(x) = tanh(λx)	Smooth, odd-symmetric (f(-x) = -f(x)), monotonically increasing. Gain at the origin, f′(0), equals λ.
Sigmoid	f(x) = 2 / (1 + e^(-λx)) - 1	Also smooth and odd-symmetric. Algebraically identical to tanh(λx/2), so its gain at the origin is λ/2: exactly half of tanh's at the same λ.
Trivalent (step)	f(x) = +1 if x > 0.001; -1 if x < -0.001; else 0	A hard threshold, not differentiable at the switching points; the discrete-signal limiting case of the smooth functions above.
None (linear)	f(x) = x	The identity: no bound and no clamp, so it is not a squashing function at all. It is offered so that the raw dynamics of the linear map can be examined (below). Growth is geometric when the map is unstable, which the runaway rule of Section 3.4 stops.

λ ("lambda") is a steepness parameter multiplying the input before it reaches f; larger λ produces a steeper, more step-like response for a smooth function. Because f′(0) is proportional to λ, λ acts as the loop gain of the linearized system around the all-zero state: for a given W, the zero state is attracting when λ is small enough and loses stability once λ is large enough for activation to sustain itself.

> **Methodological note (verified for this tool):** Because the sigmoid is tanh at half the gain, the same relationship weights can produce a genuinely different qualitative outcome, including a non-zero equilibrium under tanh decaying to exactly zero under sigmoid, purely as a function of that gain difference; doubling λ under sigmoid reproduces the tanh result exactly. This was confirmed numerically during this tool's development. The practical implication: the squashing function and λ are not interchangeable cosmetic settings, both should be reported alongside any result quoted from the tool, and λ should be varied before concluding that a model has no non-trivial equilibrium. The Baseline Equilibrium tab provides a local λ control for exactly this purpose.

Running without a squashing function. With f(x) = x and no clamp, the update is linear. In the notation of Section 3.2 it is A(t+1) = (I + W) A(t) under the relative rule and A(t+1) = W A(t) under the absolute rule; write M for the update matrix, I + W or W. With no concept locked, the state is a linear function of A(0), and the long-run behavior is set by the eigenvalues of M. If every eigenvalue has modulus below 1 (spectral radius ρ(M) < 1), activation decays to zero. If ρ(M) > 1, it grows geometrically along the dominant eigenvector, with alternating sign or rotation if that eigenvalue is negative or complex. If ρ(M) = 1, it neither decays nor grows, and in general does not converge within the iteration cap.

With a set l of concepts locked at fixed activations b and the remaining concepts free (f), the free concepts follow A_f(t+1) = M_ff A_f(t) + M_fl b. This converges to (I - M_ff)^-1 M_fl b if ρ(M_ff) < 1 and otherwise runs away. The linearization of the bounded update at the all-zero state is the same map with W scaled by λ (for tanh), which is the sense in which λ is the loop gain (above). Maps drawn with reinforcing feedback loops usually have ρ(M) > 1; the squashing function, by bounding activation, is what turns that instability into a saturated fixed point or a limit cycle.

> **Methodological note:** The bounded and the unbounded update answer different questions. The bounded update describes a map constrained to the [-1, +1] scale on which FCMs are defined; the linear update shows what the weights alone imply. Where the two disagree, the squashing function is doing real work in the model, and it should be reported alongside any result (Section 14.1). The influence and sensitivity scores of Section 6 average absolute responses from an all-zero start and need a bounded scale; when squashing is switched off they are computed with the hyperbolic tangent instead, and the tool says so.

3.4 Convergence and equilibrium

Iteration continues until one of the following stopping conditions is met:

- Convergence: the largest change in any concept's activation between consecutive iterations drops below a convergence threshold ε (default 0.001); the run has reached a fixed point, its equilibrium.
- Iteration cap: a maximum number of iterations is reached (default 100) without convergence, in which case the run is reported as non-convergent (see Section 14 on interpreting this case, including sustained oscillation).
- **Runaway (only without a squashing function).** Any activation exceeds 1000 in absolute value, or becomes non-finite. The run is stopped, reported as diverged, and its final values are the stopping point, not an equilibrium. The limit is a safeguard chosen for this tool: it keeps values and charts finite and readable, and it lies far outside the [-1, +1] scale on which the map is defined.

For every transfer function this tool offers, including the identity, f(0) = 0. Consequently, if every concept starts at exactly zero and no concept is externally fixed (see Section 9 on locking), zero is already a fixed point of the update rule and the run converges in a single iteration by mathematical necessity, independent of how the relationships are weighted. This is a property of the update rule, not a defect of any particular model, and is why a meaningful baseline requires at least some concepts to start away from zero, or a concept to be held fixed.

The Transition Point and Transition Pathway analyses (Sections 12.1 and 13.10) run each simulation on until the largest change falls below 10^-9 (or ε, if that is smaller), with at least 1000 iterations, because their transition points compare neighbouring changes of a response curve, which magnifies any residual non-convergence. Whether such a run counts as converged is still decided by ε.

> **Non-convergence caused by feedback structure:** A run that reaches the iteration cap has usually not "almost" converged: in larger maps it is typically oscillating in a sustained cycle. Under the relative rule every concept keeps its previous value and adds its incoming influence, so each concept accumulates input over time, and a balancing (negative) feedback loop then behaves like a delayed corrective signal that repeatedly overshoots. With saturating squash functions this can settle into a limit cycle in which concepts swing between values near -1 and +1. Such a cycle is a property of the loop structure rather than of the weight magnitudes: while developing the example model of version 0.2.1 (Section 14.5), scaling all weights uniformly down to half their values did not remove the oscillation, whereas removing two relationships that closed balancing loops did, and the same model converged under the absolute rule and under the sigmoid. A non-convergent result should therefore be reported as such, its trajectory inspected, and the balancing loops that pass through the oscillating concepts (Sections 5.10 and 8.6) examined, rather than the iteration cap simply being raised.

3.5 Synchronous vs. asynchronous updating

Synchronous updating computes every concept's new value from the same, single previous iteration's state, giving a deterministic, reproducible trajectory for a given model and initial state; it is the mode used by default and in every formula above. Asynchronous updating instead updates concepts one at a time in a randomly re-shuffled order at every iteration, with each update immediately seeing any already-updated values from earlier in that same pass. This is a well-documented general property of asynchronously updated discrete dynamical systems: the update order can affect both the trajectory and, for some structures, which fixed point (or whether any fixed point) is reached, so two runs of an identical model in asynchronous mode are not guaranteed to produce identical results. This tool draws that order using an unbiased Fisher–Yates shuffle at every iteration, so the order itself is drawn uniformly at random; the underlying order-dependence of asynchronous updating is inherent to the mode and not eliminated by that choice. In the Monte Carlo analysis (Section 11) the order is drawn from a seeded generator, so that a run can be repeated exactly from its seed; elsewhere the order is drawn without a seed.

3.6 Initial conditions and multiple equilibria

A nonlinear map of this kind can have more than one fixed point, in which case the equilibrium reached depends on A(0). The Baseline Equilibrium tab therefore allows A(0) to be set for the baseline run independently of the model's stored initial values: from the model's values, from a uniform value for every concept, from random values drawn uniformly from [-1, +1], or concept by concept. Running the baseline from several starting vectors and comparing the end states is the standard practical check of whether the reported equilibrium is unique for the model and settings in use. Every baseline result records the starting vector it was computed from, and the exported workbook reports it.

4. Modules and the Analysis Scope

Every analysis in the tool operates on an analysed network that may be the whole model or a part of it. This section defines that network formally, since every number reported from the tool depends on it.

4.1 Module assignment

Each concept ci carries a free-text category. Its module is defined by the normalization

$$m(ci) = \text{lowercase}(\,\text{trim}(\,\text{category}(ci)\,)\,)$$

with an empty result mapped to a single "uncategorized" module, so that category labels differing only in capitalization or surrounding spaces denote the same module. The set of modules M is the set of distinct values of m. This normalization is applied uniformly: to module views, to between-module classifications, to the category tables, and to every module-based statistic, so a relationship is classed as within or between modules by exactly the same rule everywhere in the tool.

4.2 The analysed network

An analysis scope is a triple (S, mode, isolated), where $S \subseteq M$ is the set of selected modules, mode is either "modules" or "interface", and isolated states whether concepts without any relationship in the analysed network are retained. The analysed network (C', E') is then:

$$C_S = \{\, ci : m(ci) \in S \,\}$$

$$\text{Modules mode:} \quad E_S = \{\, (ci, cj) \in E : ci, cj \in C_S \,\}$$

$$\text{Interface mode:} \quad E_S = \{\, (ci, cj) \in E : ci, cj \in C_S,\ m(ci) \neq m(cj) \,\}$$

$$C' = C_S \ \text{(isolated concepts retained)}$$

$$C' = \{\, ci \in C_S : ci \text{ has a relationship in } E_S \,\} \ \text{(isolated concepts excluded)}$$

In modules mode this is the subgraph induced by the selected modules; in interface mode it retains only the relationships that cross a module boundary (the nexus interface). The full model corresponds to S = M in modules mode with isolated concepts retained. Every method in Sections 5 to 13 is applied to (C', E') exactly as it would be to a complete model; nothing about the methods themselves changes with the scope.

4.3 Methodological consequences

- **A subsystem is analysed in isolation.** Restricting to an induced subgraph removes every relationship to concepts outside it, and with them every feedback loop that runs through another module and every influence the rest of the system exerts on the subsystem. Differences between a concept's value in a module and in the full model are therefore the effect of cutting those links: an informative comparison, not an error.
- **The interface network is deliberately artificial.** In interface mode all within-module relationships are removed. Structural measures then describe each concept's role in coupling the modules together, which is the purpose of this mode. Simulation-based results describe the dynamics of a network made of coupling relationships alone and should be used to compare coupling structures, not interpreted as predictions of system behavior.
- **Metrics that are normalized by network size** (density, betweenness, centralization, the influence and sensitivity averages) remain comparable between scopes of different size; unnormalized sums (degree, centrality, total route strength) are comparable only within a scope.

4.4 Traceability

Because a result is meaningful only together with the network it was computed on, every analysis result is tied to the analysed network at the time it was computed: changing the scope marks existing results as out of date in exactly the same way as editing the model does. Exported files carry the scope in their file names, and the Analysis Workbook opens with an Analysis Scope sheet that records S, the mode, the treatment of isolated concepts, and the numbers of concepts and relationships analysed, and states which sheets describe the complete model, which the analysed network, and which the fixed scales of Section 7.

4.5 The folded (sub-model) display

From version 0.2.1 the Network tab can fold any module into a single block, so the map can show the complete model, a module overview in which every module is folded, or one module opened with the others folded around it. Folding is a display transformation only. For a set F of folded modules, the drawn graph is obtained from the analysed network (C', E') by contracting every module in F to one node:

$$\mathrm{rep}(ci) = \text{module node of } m(ci) \ \text{ if } m(ci) \in F;\ \text{otherwise } ci$$

A relationship (ci, cj) with rep(ci) = rep(cj) lies inside a folded module and is not drawn. Every other relationship that touches a folded module is grouped with all relationships sharing the same ordered pair (rep(ci), rep(cj)), and each group is drawn as one aggregate link labelled with

$$\text{count} = |G|, \quad n+ = |\{\, e \in G : w_e \geq 0 \,\}|, \quad n- = |\{\, e \in G : w_e < 0 \,\}|, \quad \text{net weight} = \Sigma_\{e \in G\}\, w_e$$

Relationships between two unfolded concepts are drawn unchanged. A folded module is placed at the centroid of its concepts, so unfolding restores their positions; in the module overview the block positions are additionally scaled towards a common origin so the overview fills the canvas.

> **Interpretation of aggregate links:** The aggregate is descriptive. Its net weight is a signed sum of separate relationships, not an FCM weight: it can exceed 1 in magnitude, and relationships of opposite sign cancel in it, so a small net weight can hide strong but opposing couplings (the count and the n+ / n- split, shown alongside, reveal this). Aggregate links never enter a computation: every analysis in Sections 5 to 13 operates on (C', E') as defined in Section 4.2, whatever is folded. The quantitative counterparts of the overview are the category interaction matrix, the modularity and interface measures (Sections 5.7 and 5.8), and the transmission between modules (Section 8.5).

4.6 Integrity checks on opening a model

An imported model file, and the copy of the model saved in the browser, is checked before it is used, because a single malformed entry could otherwise make results undefined. The corrections are deterministic:

Problem	Correction
Concept without a name	Named "Unnamed concept k" after its position in the file.
Starting value that is not a number	Set to 0.
Starting value or weight outside [-1, +1]	Clamped to the nearest bound.
Two concepts with the same id	The second receives a new id; relationships using the id keep pointing at the first.

Problem	Correction
Relationship to a concept that does not exist	Left out.
Second relationship between the same ordered pair of concepts	Left out; the first one in the file is kept.
Relationship weight that is not a number	Set to 0 (and then reported by the Model check as a zero-weight relationship).
Scenario lock, override, or weight override referring to a concept or relationship that no longer exists, or not a number	Left out.
Simulation setting outside its valid domain	Replaced by the default value.

Every correction that changes the model's structure or values in a way the modeller would not expect (the first, second, fourth, fifth, sixth, and seventh rows) is listed in a notice when the model is opened, so it can be checked and, where results are reported, stated. Clamping, the removal of stale scenario entries, and the reset of invalid settings are applied without a separate notice. A model saved by the tool itself never needs any of these corrections; they matter for files edited by hand or produced by other software.

4.7 Automatic layout of the map

From version 0.4.5 the Network tab's Arrange menu offers four layouts built for causal maps (Readable, Systems thinking, Cluster view, and Centrality view), next to the classic layered, circular, and module-grid layouts (a classic force-directed layout was offered until version 0.4.5). Like folding, layout is a display matter only: it assigns positions to the concepts of the analysed network (C', E') and never enters a computation. It is documented here because a map is often reported as a figure, and how it was arranged affects what a reader sees in it.

Every concept i is drawn as a box of width b_i and height h_i, and receives the following metrics, computed on the network as displayed: indegree and outdegree as edge counts, their |w|-weighted versions (Section 5.1), betweenness on the directed, unweighted graph (Section 5.4; Brandes, 2001), and eigenvector centrality x on the undirected |w|-weighted graph, by power iteration on A + I (Bonacich, 1972), because on a map without cycles the directed version is zero everywhere. Two placement quantities follow from these:

$$\text{drive}(i) = (\text{outdegree_w}(i) - \text{indegree_w}(i)) \,/\, (\text{outdegree_w}(i) + \text{indegree_w}(i)), \quad \text{in } [-1, +1]$$

$$\text{centrality}(i) = 0.6\, x_i \,/\, \max_j x_j + 0.4\, \text{betweenness}(i) \,/\, \max_j \text{betweenness}(j)$$

A pure driver has drive +1 and a pure outcome -1. Strongly connected components (Tarjan, 1972) mark the feedback structures. The layout then proceeds in five steps:

- **Groups.** Louvain modularity optimization (Blondel et al., 2008) on the undirected |w|-weighted graph, visiting concepts in a fixed order, followed by the connectivity guarantee of the Leiden algorithm (Traag et al., 2019): every group is split into its connected parts. Concepts without relationships form one group. Cluster view uses the modeller's modules instead, when the model has at least two.
- **Inside each group.** A force-directed placement (Fruchterman & Reingold, 1991; implemented with d3-force, Bostock et al., 2011) over the group's own relationships, with collision on the box sizes. Mode-specific pulls are added: towards x = -drive(i) times the group's spread (strong in Systems thinking, weak in Readable), towards the group's centre in proportion to centrality(i) (strong in Centrality view), and, in Readable and Systems

thinking, onto a ring for every feedback component of three to seven concepts, so that a short loop is drawn as a circle.

- **Between groups.** Each group becomes a disc large enough to hold its layout. The discs are placed by a force-directed layout of the group graph, in which two groups are linked with a strength proportional to the summed |w| of the relationships between them, and are then separated until none overlap. Systems thinking pulls groups with a high mean drive to the left, Centrality view pulls groups with a high mean centrality to the middle, and Cluster view widens the gaps.
- **Assembly.** Each group is rotated (twelve angles) and reflected to the orientation that minimizes the total length of its relationships to other groups; Systems thinking only reflects top to bottom, so the left-to-right order is kept. A short global force pass then lets concepts drift towards their partners in other groups while staying near their group's position.
- **Local search.** Overlapping boxes are pushed apart, and every concept that has a problem (a crossing, a line through another box, an overlap, or a clashing label) is tried at sixteen nearby positions, at the barycentre of its neighbours and halfway to it, and swapped with its six nearest same-group concepts of similar size. A move is kept when it lowers the concept's share of the objective below.

The objective, which is also reported for the finished layout, is

> score = -10 X - 5 O_n - 5 O_l - 2 L + 3 S + 3 V_c + 2 V_h

where X counts pairs of crossing relationship lines (lines sharing an end excluded) plus lines that pass through a box other than their own two; O_n counts overlapping boxes; O_l counts weight labels, assumed at the middle of their line, that overlap a box; L is the mean line length in units of the mean box width plus 60 pixels; S is the mean distance from each group's centre to the nearest other group's centre, divided by twice the mean distance of concepts from their own group's centre (capped at 3); V_c is the centrality-weighted mean of 1 - r_i / r_max, where r_i is a concept's distance from the middle of the map; and V_h is the correlation between drive(i) and position towards the top left. A reciprocal pair counts as one line, because it is drawn as two lines bowed apart. In Systems thinking the local search also penalizes a concept's horizontal distance from its drive-based position, and in Centrality view a central concept's distance from the middle.

Up to three candidate layouts are built (one anchored firmly to the group plan, one looser, and in Readable one left free to settle as a plain force-directed layout from the group plan) and the one with the highest score is kept. The local search runs for a fixed amount of work, not for a fixed time, so the same model always gives the same arrangement; a time limit only stops it on an unusually slow computer. The computation runs in a separate browser thread, and the cards are animated to their new positions.

In the Figure style, two further display steps follow every layout, automatic or by hand. A relationship that would run through another box is bowed, by the smallest of a set of offsets (40 to 190 pixels, either side) that clears every box, and a reciprocal pair is bowed apart. Each weight label is then placed at the fraction of its line (0.5, then 0.42 or 0.58, and so on out to 0.27 or 0.73) where it overlaps the fewest boxes and already placed labels, the strongest relationships choosing first.

What the layout does and does not mean: The weights in the objective are design choices of this tool, set to follow the usual priority in the graph-drawing literature, which finds edge crossings to be the aesthetic that most affects how well a drawing is understood (Purchase, 1997). Groups found by the layout are a visual aid, not a result: they are recomputed from the drawn relationships each time and are not the modules of Section 4.1, except in Cluster view. Positions carry no meaning beyond the stated tendencies (drivers left in Systems thinking, central concepts in the middle in Centrality view), and every analysis is independent of them. In Systems thinking, keeping drivers left and outcomes right forces feedback that runs back from an outcome to a driver to cross the map, so this mode trades some crossings for its left-to-right reading.

4.8 Figure export

Figures are exported so that their text prints at a chosen size t (in points) at a chosen width. A figure that lays itself out from the width of its container is drawn at a layout width w = W (f × 0.75) / t, where W is the target width in CSS pixels at 96 per inch, f the figure's typical font size in pixels, and 0.75 the conversion from pixels to points, and the result is then scaled to W. Its labels are therefore placed by the figure's own layout rules at the size they will have, instead of being stretched. Figures drawn at a fixed aspect ratio (the metro map and the transition point panels) have their font sizes scaled by t / (0.75 f) instead.

A change of palette is applied to the exported copy only. Colors of similar hue are grouped into families, each family is given the nearest palette color not yet used, and lighter shades within a family stay lighter, so that the distinctions of the original figure survive. Greys and near-black text are kept as they are. The greyscale option converts every color to its luminance.

5. Structural (Graph-Theoretic) Analysis

The methods in this section are computed directly from the weight matrix W of the analysed network and require no simulation: they describe the map's structure as drawn, independent of how it behaves dynamically (Section 6 covers the dynamic counterpart).

5.1 Degree-based centrality

For each concept, indegree and outdegree are the edge-weighted analogues of standard graph-theoretic degree centrality: the sum of absolute incoming/outgoing relationship weights, rather than a simple edge count, so a single strong relationship and several weak ones can be compared on the same footing:

$$\text{indegree}(ci) = \Sigma j\ |wji|$$

$$\text{outdegree}(ci) = \Sigma j\ |wij|$$

$$\text{centrality}(ci) = \text{indegree}(ci) + \text{outdegree}(ci)$$

$$\text{net influence}(ci) = \text{outdegree}(ci) - \text{indegree}(ci)$$

5.2 Driver and receiver scores

To express a concept's balance between sending and receiving influence independently of how strongly connected it is overall, the tool reports

> $\text{driver score}(ci) = \text{outdegree}(ci) \,/\, \text{centrality}(ci)$

> $\text{receiver score}(ci) = \text{indegree}(ci) \,/\, \text{centrality}(ci)$

both defined as 0 for an isolated concept. The two sum to 1. These are straightforward normalizations of the degree figures in Section 5.1, reported by this tool as a convenience; they are not a named index from a specific source.

5.3 Role classification

This tool classifies every concept into one of five structural roles, using thresholds set for this implementation rather than drawn from a single external source:

Role	Rule
Isolated	centrality(ci) = 0
Hub	centrality(ci) is at or above the 75th percentile of the non-zero centralities in the analysed network, AND indegree(ci) > 0 AND outdegree(ci) > 0
Driver	(not already Hub) outdegree(ci) > 1.2 × indegree(ci)
Receiver	(not already Hub) indegree(ci) > 1.2 × outdegree(ci)
Connector	everything else

The underlying vocabulary (driver, receiver, hub) is standard in the FCM and social-network-analysis literature broadly (see e.g. the survey by Papageorgiou & Salmeron, 2013); the specific 75th-percentile and 1.2× cut points that operationalize it in this tool are an implementation choice, not a value reported in that or any other single source, and should be reported as such if a role classification from this tool is quoted.

5.4 Betweenness centrality

Betweenness centrality measures how often a concept lies on the shortest routes between other concepts (Freeman, 1977). For the directed analysed network, with σst the number of shortest directed paths from s to t and σst(ci) the number of those passing through ci,

> $\text{betweenness}(ci) = [\ \Sigma_(s \neq ci \neq t)\ \sigma st(ci) \,/\, \sigma st\ ] \,/\, ((n-1)(n-2))$

computed with Brandes' (2001) algorithm. The normalization by (n-1)(n-2), the number of ordered pairs a concept could lie between, bounds the measure in [0, 1] and keeps it comparable between networks of different size. Two implementation choices apply and should be reported with any betweenness value from this tool: shortest paths are computed on the unweighted graph (each relationship counts as one step, whatever its weight), and parallel relationships and self-loops are ignored. The first choice is deliberate: FCM weights express strength of influence, not distance, and there is no agreed transformation from one to the other, so betweenness here measures brokerage position alone.

5.5 Density, connectivity, and centralization

Statistic	Formula
Density	(actual relationships ÷ n(n-1)) × 100%
Average degree	(2 × relationships) ÷ n
Connectivity	relationships ÷ n
Centralization	(Σi (max centrality - centrality(ci))) ÷ ((n-1) × max centrality) × 100%

These are standard descriptive network statistics; centralization follows the general construction of Freeman (1978), comparing every node's centrality to the network's most central node and normalizing by the maximum such total gap, here using the weighted degree centrality of Section 5.1.

5.6 Clustering coefficient

The clustering coefficient is computed on the undirected projection of the analysed network (ci and cj are neighbors if a relationship exists in either direction). For a concept with ki ≥ 2 neighbors, its local coefficient is the fraction of pairs of its neighbors that are themselves neighbors, and the network value is the mean over all concepts with ki ≥ 2 (Watts & Strogatz, 1998):

> $Ci = (\text{links among the neighbors of ci}) \,/\, (\, ki\ (ki - 1) \,/\, 2\,)$

> $C = \text{mean of Ci over the concepts with ki} \geq 2$

Weights and directions are not used. The number of concepts the mean is taken over is reported with the value, since concepts with fewer than two neighbors are excluded by construction.

5.7 Modularity of the module partition

To express how well the modules defined by the modeller correspond to the network's structure, the tool reports the Newman–Girvan modularity Q of the partition into modules (Newman & Girvan, 2004), computed on the undirected projection with edge weights |wij| (a pair of reciprocal relationships contributes both weights):

> $Q = \Sigma_(k \in M)\ [\ Lk \,/\, m - (\, dk \,/\, 2m\,)^2\ ]$

where m is the total absolute weight of all relationships in the analysed network, Lk the total absolute weight of relationships inside module k, and dk the total weighted degree of the concepts in module k. Clearly positive values mean that the modules are more tightly connected internally than would be expected by chance given the concepts' degrees; values near zero mean that the module boundaries do not follow the network's structure; negative values mean that they cut across it. Q is undefined, and reported as not available, with fewer than two modules or no relationships in scope. Two points should accompany any reported value: this is the modularity of a given partition (the modeller's modules), not the result of a community-detection search for the partition that maximizes Q; and its use of absolute weights means negative relationships count as connection strength, like the degree measures of Section 5.1.

5.8 Interface relationships

The number of relationships whose two concepts belong to different modules, and their share of all relationships in the analysed network, is reported as a direct measure of how strongly the subsystems in scope are coupled. It is a simple count under the module definition of Section 4.1.

5.9 Structural complexity index

Structural complexity is reported as the ratio of Receiver-role to Driver-role concepts: the Özesmi & Özesmi (2004) FCM complexity index, adopted here without modification. Higher values indicate a model that leans "outcome-heavy" (more receivers than drivers, common in models emphasizing consequences); lower values indicate a "driver-heavy" model emphasizing levers and causes (Özesmi & Özesmi, 2004). Because this ratio depends on the same 1.2× role threshold described in Section 5.3, a model right at that threshold can shift substantially in complexity from a small edit, worth keeping in mind when comparing complexity across model versions.

5.10 Feedback loop detection and classification

Simple cycles (feedback loops) up to eight relationships long are enumerated by depth-first search from every concept in the analysed network, each cycle counted once (from its lexicographically smallest concept), capped at 300 found loops to keep the search tractable on dense networks. Each loop is classified as reinforcing (amplifying) or balancing (dampening) by the parity of negative relationships it contains: an even number of negative relationships around the loop is classified reinforcing, an odd number balancing.

The underlying distinction, between mutually causal processes that amplify a deviation and those that counteract it, is due to Maruyama's (1963) "second cybernetics" (Maruyama, 1963). The even/odd-negative-edge parity rule used to operationalize that distinction here is a standard convention in signed-graph and cognitive-mapping analysis for determining a cycle's net sign; it is not itself part of Maruyama's original formulation, which predates the fuzzy cognitive map formalism. The breakdown of the same loops by module is described in Section 8.6.

6. Perturbation-Based Influence and Sensitivity Scores

6.1 Rationale

The structural metrics in Section 5 describe the map as drawn, but a concept's static degree does not necessarily predict how much it actually moves the rest of the system once feedback loops and indirect paths are engaged by simulation: two concepts with identical outdegree can propagate very differently depending on the structure downstream of them. This tool complements the static metrics with a dynamic, simulation-based pair of scores, obtained by perturbing the model and observing the propagated response: a one-factor-at-a-time (OAT) perturbation design, a standard family of approaches in the broader sensitivity-analysis literature (Saltelli et al., 2007).

6.2 Formal procedure

For every concept ci in the analysed network, in turn:

1. Every concept is set to activation 0.
2. ci alone is fixed (locked) at activation +1 for the entire run.

3. The model is simulated to equilibrium under the current squash function, λ, update rule, and convergence settings (Section 3); when squashing is switched off, the hyperbolic tangent is used instead, since the scores average absolute responses and need a bounded scale.
4. For every other concept cj, the absolute final activation |Aj_final| is recorded.

> $\text{influence}(ci) = (1/(n-1)) \cdot \Sigma_{(j \neq i)} |Aj_final \text{ under driven run } i|$

> $\text{sensitivity}(cj) = (1/(n-1)) \cdot \Sigma_{(i \neq j)} |Aj_final \text{ under driven run } i|$

Influence(ci) is thus the average movement ci causes in every other concept across its own driven run; sensitivity(cj) is the average movement cj undergoes, collected across every other concept's driven run. Because every run starts from an explicit, forced all-zero state (every concept but the one being driven), these scores are independent of whatever initial values are configured on the model for other purposes; they measure the network's propagation structure in isolation.

6.3 A known limitation of this design

Like any one-factor-at-a-time perturbation design, this procedure does not capture interaction effects between concepts: cases where the joint effect of driving two concepts together differs from the sum of their individually measured effects. This is a well-documented general limitation of OAT approaches relative to designs that vary multiple factors jointly (Saltelli et al., 2007). Where interaction effects are of interest, the Scenarios & Simulation tab (Section 9) allows a scenario with multiple concurrent interventions to be run and its combined effect compared directly against the individual interventions' effects.

7. Multi-Scale Analysis

The multi-scale analysis compares every concept's importance at three fixed scales, independent of the analysis scope selected elsewhere in the tool. It is an original analysis developed for this tool, built entirely from the methods in Sections 4 to 6.

7.1 Scales

For a model with modules M, the scales are the following analysed networks (Section 4.2):

- **Global:** the full model (S = M, modules mode, isolated concepts retained).
- **Own module:** for each module k, the subgraph induced by k alone (S = {k}); each concept is scored in the network of its own module.
- **Interface:** every module in interface mode (S = M, only between-module relationships), available when there are at least two modules; isolated concepts are retained, so a concept without any between-module relationship is scored 0 there.

At each scale the structural metrics of Section 5 and the perturbation scores of Section 6 are recomputed from scratch on that scale's network, with the same simulation settings. A value at one scale is therefore never a slice of another scale's value.

7.2 Ranking

Within each scale, concepts are ranked on a chosen metric (influence score, centrality, betweenness, or out-degree) using standard competition ranking: sorted in descending order, with tied values sharing the better rank (the "1224" convention). Ranks are only comparable as positions within a scale, not as raw numbers across scales of different size; for comparison the tool converts a rank r among n concepts to a percentile position

> $p = 1 - (r - 1) / (n - 1)$ $(p = 1 \text{ if } n = 1; p = 0 \text{ if the value at that scale is } 0)$

so that a concept with no position at a scale (for example no between-module relationship) counts as bottom there rather than tied for first. The own-module percentile is left undefined for modules with a single concept.

7.3 "Importance mainly from"

To summarize where a concept's importance comes from, the tool compares its percentiles at the defined scales with the following rule, applied in order:

Result	Rule
Peripheral at every scale	Every defined percentile is below 0.5.
Important at every scale	At least two scales are defined and every percentile is at least 0.75.
Mixed	The highest percentile exceeds the second highest by less than 0.15.
System-wide position / Within-module dynamics / Bridging between modules	Otherwise, the scale (global / own module / interface) with the highest percentile.

> **Status of this rule:** The thresholds 0.5, 0.75, and the margin 0.15 are conventions chosen for this tool, not values from the literature. The label is a reading aid for navigating the ranks; if the classification of a concept matters to an argument, the underlying ranks should be reported instead.

7.4 Leverage points by scale

For each scale, the tool lists the concepts with the highest influence score (Section 6), excluding concepts with an influence of zero and breaking ties by out-degree: the top three on screen, the top five in the exported workbook. "Leverage point" is used here in the operational sense of the concept whose sustained activation moves the rest of that scale's network the most on average; each scale is ranked independently, and the rankings are expected to differ.

7.5 Limitations

- The analysis inherits the limitations of its components: OAT perturbation (Section 6.3) and the artificial character of the interface network (Section 4.3).
- Its computational cost is one simulation run per concept at each scale (approximately three runs per concept in total), which on large models takes a noticeable time; the result is computed after the tab has opened and cached until the model's structure or settings change.

8. Influence Routes and Feedback Loops

This section describes the structural route analysis of the Influence Routes tab: how indirect influence can travel through the analysed network, which concepts that transmission depends on, and how feedback loops divide between and within modules. These are structural analyses: they require no simulation.

8.1 Routes and their enumeration

An influence route is a simple directed path in the analysed network: a sequence of concepts c0 → c1 → … → cL connected by relationships, in which no concept appears twice. Its length L is the number of relationships. The tool enumerates routes by depth-first search from every eligible start concept, for lengths between a chosen minimum and maximum (at most 6). Enumeration stops after 250,000 search steps or 20,000 retained routes, whichever comes first; if either limit is reached, the result is flagged and every derived count and share is based on the routes found before the limit.

8.2 Strength and sign

> $\text{strength}(\text{route}) = \Pi_(l=1..L)\ |w(c_(l-1), c_l)|$

> $\text{sign}(\text{route}) = \Pi_(l=1..L)\ \text{sign}(\ w(c_(l-1), c_l)\ )$

The strength of a route is the product of the absolute weights along it, so a route is never stronger than its weakest relationship and strength decays with length; its sign indicates whether the first concept pushes the last in the same (+) or the opposite (-) direction. Kosko (1986) defined the indirect effect of a causal path through its weakest link (a minimum operator) and the total effect between two concepts through the strongest such path (a maximum operator). The product used here is an alternative choice of combining operator, made for this tool so that longer chains of equally strong relationships are weaker than shorter ones; routes are reported individually and summed only in the aggregate measures of Section 8.4.

8.3 Route types

With m(·) the module function of Section 4.1, a route is included in a selection when:

Route type	Inclusion rule
All routes	Every route.
Within module k	Every concept on the route belongs to k.
Between modules a and b	The route starts in a and ends in b (direction a→b), starts in b and ends in a (b→a), or either (both directions); intermediate concepts may belong to any module in scope.
Cross-module (Nexus Interface)	The route contains at least one relationship (c_(l-1), c_l) with m(c_(l-1)) ≠ m(c_l).

A selection may additionally be restricted to a given start concept and/or end concept.

8.4 Bridge concepts, boundary crossings, and bottlenecks

For the set R of routes in a selection, the tool reports for every concept c:

> $\text{share}(c) = \Sigma_(r \in R,\ c \text{ intermediate in } r)\ \text{strength}(r)\ /\ \Sigma_(r \in R)\ \text{strength}(r)$

where "intermediate" excludes a route's first and last concepts; the number of routes passing through c; and its boundary crossings, the number of times a route in R changes module at c (each between-module relationship on a route counts once for each of its two concepts). Concepts are listed in descending order of share. A concept is flagged as a bottleneck when share(c) ≥ 0.5, meaning that at least half of the combined strength of the selected routes passes through it.

> **Status of these measures:** Route-based brokerage of this kind is related in spirit to betweenness centrality (Section 5.4), which counts shortest paths only; the strength-weighted share over all simple routes of bounded length, the boundary-crossing count, and the 50% bottleneck threshold are this tool's own constructions and should be reported as such.

8.5 Transmission between modules

Routes in a selection are also grouped by the pair (module of the first concept, module of the last concept). For each pair the tool reports the number of routes, their combined strength, and the numbers of positive and negative routes, summarizing along which module-to-module channels indirect influence can be transmitted.

8.6 Feedback loops by module

The feedback loops of Section 5.10 (same enumeration and limits, so the totals are identical) are further classified by the modules of their concepts. A loop is within a module if every concept on it belongs to the same module, and between modules otherwise. The tool reports the numbers of reinforcing and balancing loops for the whole analysed network, within modules (per module), and between modules (per combination of modules a loop passes through); for each module, both the loops entirely inside it and all loops passing through it; and every loop individually with its strength, the product of the absolute weights around it. Between-module loops are the feedback processes that couple subsystems: a change in one module returns to it only after passing through another. In interface mode (Section 4.2) all within-module relationships are removed, so no within-module loop can exist there.

8.7 Limitations

- Routes and loops describe where influence can travel given the structure, not how much of an initial change actually arrives once the nonlinear update rule and competing feedback act on it; that is what the simulation-based analyses measure.
- Enumeration is bounded (routes up to 6 relationships, loops up to 8, with the caps given above); on dense networks the reported sets can be incomplete, which the tool flags.
- Aggregate route measures (share, combined strength) depend on the chosen length bounds, since longer routes add many weak paths; they should be reported together with the selection they were computed for, which the exported workbook records.

9. Scenario Analysis Methodology

9.1 The driver/lock formalism

A scenario intervention is applied by fixing (locking) one or more concepts at a specified activation for the entire simulation run, most commonly full activation, ±1, representing a maximal policy lever or shock, while the rest of the model continues to evolve under the ordinary update rule. This "driver" convention, in which a concept is forced to a fixed extreme value and held there for the whole run to represent a policy intervention, follows the scenario-analysis approach implemented in Mental Modeler, a widely used participatory FCM tool (Gray et al., 2013), and is used here in the same form: locking is applied both to the initial state and throughout every subsequent iteration, so the intervention is a sustained condition rather than only a starting point that the system's own dynamics could otherwise erode.

A dedicated Baseline Equilibrium run (Section 3.4) with no concept locked provides the fixed reference point every scenario is compared against; a scenario's equilibrium is computed by the same procedure with one or more concepts locked. Scenarios are simulated on the analysed network of the current scope (Section 4).

9.2 Effect size, synergy, and trade-off

These comparative measures are this tool's own operational definitions, applied to scenario-versus-baseline equilibrium differences:

> $\text{effect size}(ci, \text{scenario}) = \text{Ai_final}(\text{scenario}) - \text{Ai_final}(\text{baseline})$

When two or more outcome concepts are evaluated under a scenario, their effects are jointly classified as synergy (a majority-to-all improve), trade-off (some improve while others worsen), or negative synergy (a majority-to-all worsen), each further graded full/partial/narrow by what share of the evaluated outcomes moved in that direction, and against a flat-response threshold below which a concept is treated as effectively unmoved. This terminology is common in the policy-scenario literature generally; the specific share-based grading rule is an implementation choice made for this tool.

10. Sensitivity Analysis (Dedicated Tab)

Where Section 6's scores summarize influence and sensitivity as a single perturbation to a fixed extreme, this tab performs a full local sensitivity sweep: one input concept is locked, in turn, at every value across a user-chosen range [vmin, vmax] sampled at a chosen resolution, the model is re-simulated at each value, and the equilibrium of one or more chosen output concepts is recorded at each step; the standard range-sweep design used throughout the simulation and decision-analysis literature for one-dimensional local sensitivity analysis.

10.1 Elasticity

For each output concept, elasticity is reported as the average slope of its response across the full swept range:

> $\text{elasticity}(\text{output}) = (\text{Aoutput at vmax} - \text{Aoutput at vmin}) \,/\, (\text{vmax} - \text{vmin})$

This is the standard finite-difference (secant) definition: output change per unit input change, averaged across the sampled range rather than evaluated at a single point. Because only the two endpoints enter the formula, a

response that rises and then falls can report an elasticity near zero while moving considerably; it should always be read against the response curve.

10.2 Tornado ranking

Independently of the chosen input concept, every other concept in the model is swept across the same [vmin, vmax] range one at a time (holding it fixed at each end value while every other concept evolves under the ordinary update rule), and ranked by how much it moves the primary output. This produces a tornado diagram, a standard visualization in decision and risk analysis for ranking the relative importance of a model's inputs by the size of their effect on an output, identifying which concept the chosen output is actually most sensitive to, which need not be the concept originally selected as input.

11. Monte Carlo Simulation

This analysis, new in version 0.4.0, applies the Monte Carlo method, the propagation of input uncertainty through a model by repeated random sampling (Metropolis & Ulam, 1949), to the weights of a fuzzy cognitive map. It complements the deterministic analyses of Sections 9, 10, 12, and 13: those report where the map settles for the weights as recorded, and this one reports how far that answer depends on the weights being exactly as recorded. Its motivation is the status of the weights themselves (Section 14.1): they are elicited beliefs, not measured parameters, so an equilibrium computed from them carries an uncertainty that the deterministic analyses do not show. The method is standard in uncertainty and sensitivity analysis (Saltelli et al., 2007); the sampling design below is this tool's own.

11.1 Sampling design

A scenario fixes a set of locked concepts, a set of overridden weights, and a set of overridden initial values (Section 9.1). For run r = 1, ..., R, every relationship weight that the scenario does not override is replaced by

> $w'ij(r) = \mathrm{clip}(\, wij + uij(r) \cdot \delta w \,, -1, +1 \,), \quad uij(r) \sim U(-1, +1)$, independent

where δw ≥ 0 is the weight uncertainty (default 0.1), clip restricts the value to [-1, +1], and the result is rounded to two decimals, the resolution at which the tool stores weights. If the starting-value uncertainty δa is greater than zero, every initial value that the scenario does not override is perturbed in the same way, ci(r)(0) = clip(ci(0) + vi(r) · δa , -1, +1). Locked concepts, overridden weights, and overridden initial values are never perturbed: a scenario that pins a quantity states it as a deliberate "what if this were exactly x", not as an uncertain estimate. Each perturbed model is then simulated by the ordinary procedure (Sections 3.2 to 3.5) under the current settings, and the final state A(r) and whether the run converged are recorded. A nominal run, with no perturbation, is simulated alongside.

11.2 Summary statistics

For every concept i, the tool reports over the R final values Ai(r): the nominal value, the mean, the median, the standard deviation (with divisor R, that is, the spread of the sample itself), the minimum and the maximum, and the percentiles P5, P25, P75, and P95. Percentiles are taken by nearest rank on the sorted values: Pp is the value at position round((p/100)(R - 1)), counting from zero. Percentiles serve as the plausible range in place of a normal approximation, because distributions of equilibria are often skewed, bounded by ±1, or bimodal when the map is multistable (Section 3.6). The tool also shows the sorted sample as a histogram, so that bimodality can be seen and not only summarized.

The number of runs that converged is reported and, without a squashing function, the number that ran away (Section 3.4). The standard error of a reported mean is approximately σ/√R, so with the default 500 runs it is about 4.5% of the standard deviation; the number of runs can be raised to 2,000.

11.3 Randomness and reproducibility

Sampling uses mulberry32, a small 32-bit pseudo-random generator, seeded with the user's seed (by default a random number below one million, which is displayed so that it can be recorded). The same model, settings, and seed reproduce every sampled weight, and therefore the whole result, exactly. In asynchronous mode (Section 3.5) the random update order is drawn from a second stream seeded from the same seed, and the nominal run from a third, so that the result is reproducible in that mode as well and the sequence of sampled weights does not depend on the update mode. The generator is adequate for sampling noise and is not intended for security purposes, which are not at issue here.

11.4 Interpretation and limitations

- **A stated spread, not an estimate.** δw is an assumption about how uncertain the weights are; the tool does not estimate it. Choosing it is the analyst's task. On the tool's own scale of preset strengths (0.25 apart: weak, moderate, strong, very strong), a spread of 0.25 represents each weight being one step off and 0.1 a smaller disagreement. Reporting results at several spreads, for example 0.1 and 0.25, shows how quickly a conclusion degrades.
- **Independent, uniform, and equal.** Real disagreement is not independent across weights (an expert who is optimistic about one relationship may be about several) and not equal (some weights are better founded than others). Per-relationship spreads and correlated draws are not offered.
- **Clipping.** A weight near ±1 can only move inward, so its mean shifts slightly inward; this follows from the bounded scale.
- **Structure is fixed.** Only weights and starting values are sampled. Relationships are not added, removed, or reversed, and the module assignment does not change; structural uncertainty is examined by editing the map.
- **Runs that do not settle.** Runs that reach the iteration cap or run away are included in the statistics and flagged. In a map that settles slowly or lies near an oscillation, perturbation can move a run over the cap although the nominal run settles (Section 14.3, and the example of Section 14.5); the share of such runs is a finding about the map, and raising the cap removes it only in part.
- **A distribution of model equilibria, not a probability.** The output is the distribution of equilibria under an assumed distribution of weights. It is not the probability of a real-world outcome (Section 14.4).
- **Relation to the other designs.** Monte Carlo varies all weights together and at random; the one-at-a-time designs of Sections 6 and 10 vary one factor deliberately and find leverage points. The tool does not compute variance-based sensitivity indices from the sample, so the summary shows which concepts are sensitive to the joint uncertainty, not which weights are responsible.
- **Plain-language reading.** The box What these results mean classifies a run by fixed rules on the 5 to 95 percent range (P95 - P5) of each concept. A concept is firm when that range is below 0.1 and uncertain when it is 0.3 or wider; it could go either way when P5 < -0.05 and P95 > 0.05. The picture is fragile when fewer than 90% of the runs settle, robust when at least 70% of the concepts are firm and none could go either way, mostly robust when at least 40% are firm and at most two could go either way, and uncertain otherwise. These cut-offs are conventions of the tool that make the table readable; they are not statistical tests.

12. Transition Point Analysis

This analysis is an original method developed for this tool; it has no single external literature source and is documented here in full so its assumptions and limitations are available alongside any result derived from it. It is motivated by marginal-analysis reasoning common in intervention-effectiveness and dose-response research generally: for a graded intervention, the question of practical interest is often not only "does it work?" but "at what intensity does additional effort stop paying off at the same rate?"

12.1 Procedure

One intervention concept is swept in n equal steps, from 0 up to +1 (an increase) or from 0 down to -1 (a decrease), locking it at each step value and re-simulating the model to equilibrium, exactly as in Section 10 but restricted to the range [0, 1] or [-1, 0] with a fixed direction. These are the two ways a scenario drives a concept, held at +1 or at -1 (Section 10): both sweeps start with the concept held at 0, and the last step of each is the scenario in which it is held at +1 or -1. The steps are reported as an intensity x between 0 and 1, so that the concept is held at x in an increase and at -x in a decrease. For each chosen outcome concept, the discrete marginal benefit of every step is computed, oriented by the sign s of the outcome's total effect (s = +1 if the outcome rises over the sweep, -1 if it falls):

$$MB(i) = s \cdot (Aoutput(x(i+1)) - Aoutput(x(i))) / \Delta x, \quad i = 0, ..., n-1$$

The transition point is the sampled intensity at which the marginal benefit declines most steeply, the point of diminishing returns:

$$\text{transition point} = x(i^*), \quad i^* = \text{argmin over } i = 1, ..., n-1 \text{ of } (MB(i) - MB(i-1)) / \Delta x$$

If the steepest decline falls on the last interior level (i* = n - 1), or the marginal benefit does not decline anywhere in the range, the transition point is reported as 1: the response has no point of diminishing returns inside the range (Proportional, Section 12.2). A decline smaller than 1% of the largest |MB(i)| per unit of intensity counts as no decline. Second differences amplify numerical noise, so every run of the sweep continues until no concept changes by more than 10^-9 per iteration (or the global threshold, if smaller), with at least 1000 iterations (or the global limit, if higher); whether a run counts as converged still follows the global threshold (Section 3.4), and the number of runs that did not reach 10^-9 is reported. This definition replaces that of version 0.4.2 and earlier, the step with the largest |slope|, which marks where a response is steepest rather than where its returns start to diminish.

Changes are taken with respect to the intensity x, so for a decrease a positive change means that the outcome rises as the concept is pushed further down. The total effect used in Section 12.3 is the change from the concept held at 0 to the concept held at +1 or -1. The reference state is therefore the concept held at 0, which is not the baseline equilibrium in which the concept is free to move.

> **Methodological limitation (by design, stated explicitly):** This tool has no per-concept notion of which direction constitutes an improvement, so benefit is counted in the direction in which the outcome moves over the whole sweep (s above): for a falling outcome, a further fall is its marginal benefit, and for an outcome that first moves one way and then back, the orientation follows the net effect. A transition point on its own therefore does not say whether the change it marks is desirable: the sign of the accompanying effect size must always be checked before treating a transition point as a leverage point worth pursuing. Transition Pathway Analysis (Section 13) takes the other route: the analyst states the desirable direction of every outcome, and its transition points are defined on that signed basis.

12.2 Response-type classification

Each outcome is classified by where, in the [0,1] range, its transition point falls, checked in this order:

Type	Condition
Flat	Total range of the outcome across the whole sweep is below a flat-response threshold; checked first, wherever the marginal benefit happens to decline.
Proportional	Transition point 1 (no decline of the marginal benefit before full intensity, Section 12.1).
Early	Transition point at most 1/3.
Middle	Transition point between 1/3 and 2/3.
Late	Transition point of 2/3 or more, below 1.

This equal-thirds operationalization is an implementation choice for this tool; the early/late vocabulary of diminishing returns is used descriptively, as in dose-response and intervention analysis generally, without reference to one specific formal source.

12.3 Efficiency

An outcome's efficiency is reported as its total effect (final value minus initial value across the full sweep) divided by its transition point:

$$\text{efficiency(outcome)} = (\text{Aoutput}(1) - \text{Aoutput}(0)) \,/\, \text{transition point}$$

This is intended as a rough "how much change, how early" measure: a high value indicates most of an outcome's total change is achieved with comparatively little of the intervention's range spent, a low value that most of the range is spent for comparatively little total change. It is this tool's own construct, not a standard sensitivity-analysis statistic, and its sign follows the sign of the total effect (a large negative total effect produces a large-magnitude negative efficiency, not a "good" one).

The transition point is never below the first sampled step, so the efficiency is always defined. The results also report the change at the transition point, Aoutput(TP) - Aoutput(0), and its share of the total effect, (Aoutput(TP) - Aoutput(0)) / (Aoutput(1) - Aoutput(0)): how much of the effect has been reached by the point of diminishing returns. The share is not reported when the total effect is below the flat-response threshold, and it exceeds 100% when the outcome overshoots its final value.

The prioritization matrix of the tab places every outcome that is not Flat by its transition point and the absolute value of its total effect, |Aoutput(1) - Aoutput(0)|. It splits the plane at a transition point of 0.5 and at the mean absolute effect of the outcomes shown, giving four quadrants: Quick wins (effect at or above the mean, transition point at most 0.5), Strategic investments (effect at or above the mean, transition point above 0.5), Small improvements (below the mean, at most 0.5), and Poor leverage points (below the mean, above 0.5). The split is relative to the outcomes analyzed together, so a quadrant describes an outcome compared with the others, not an absolute standard.

13. Transition Pathway Analysis

This analysis, new in version 0.3.0, is an original method developed for this tool. It extends Transition Point Analysis (Section 12) from single levers to ordered sequences of levers and is documented here in full. Its motivation comes from literatures that treat transformation as a staged and path-dependent process: resilience thinking describes transformation as movement between stability domains of a system's state space (Walker et al., 2004); increasing returns make the value of later options depend on earlier choices (Arthur, 1989); adaptation pathways formalize policy as sequences of actions with explicit switching conditions (Haasnoot et al., 2013); and the leverage points perspective stresses that interventions interact, shallow ones creating the conditions for deeper ones (Abson et al., 2017; Fischer & Riechers, 2019). These sources motivate the method; the operational definitions, thresholds, and rules below are this tool's own.

In an FCM, a single intervention moves the equilibrium from one point of the state space to another, and a sequence of interventions produces a trajectory of equilibria. The method evaluates such trajectories: each stage implements one lever up to its transition point in the state reached by the preceding stages, so the analysis registers how interventions enable or constrain one another.

13.1 Outcomes, desirability, and outcome gain

Let O be the set of nexus outcome concepts chosen by the analyst. Each outcome k carries a desirability direction dk ∈ {-1, +1} (lower or higher is better), a weight ωk ≥ 0 (default 1), and optionally a desired level. The Business-as-Usual state A(BAU) is the equilibrium reached from the model's stored initial values with no concept locked. The outcome gain of a state A, relative to BAU, is

$$G(A) = \Sigma k \in O \;\; \omega k \cdot dk \cdot (Ak - Ak(BAU))$$

Because dk orients every outcome, G measures improvement. This is the key difference from Section 12, which counts benefit in whichever direction an outcome moves: here the analyst states the desirable direction, so the transition point marks where improvement starts to diminish.

13.2 Levers and the equilibrium operator

A lever ℓ is operated at an intensity u ∈ [0, 1]; its activation is u for an increasing lever and -u for a decreasing one (so that intensity 0 is the neutral lever in both directions, and intensity 1 is the lever held at +1 or -1, as in Section 12.1), and it carries an effort weight cℓ > 0 (default 1). A lock set L holds levers at fixed activations for a whole run. Φ(A0, L) denotes the equilibrium operator: the state reached by iterating the update rule (Section 3.2) from the state A0 with the levers in L held fixed, until the largest change in any activation falls below 10^-9, or the convergence threshold if that is smaller (Section 13.10). Pathway runs are always synchronous (Section 3.5), so every pathway is deterministic and reproducible, and they use their own iteration cap (Section 13.10); every other setting (squash function, λ, update rule, and the convergence threshold that decides whether a run counts as converged) is the global one. Φ is implemented as a compact typed-array version of the tool's standard simulation for speed; the test suite checks that both give identical final states (difference 0 to machine precision) under tanh and sigmoid, under the relative and absolute rules, and from a warm start.

13.3 Stages and conditional transition points

A pathway of length J is a sequence of stages P = ((ℓ1, u1), …, (ℓJ, uJ)). With L0 = ∅ and A(0) = A(BAU), stage j sweeps its lever over the grid xi = i/N (N = 10, 20, or 50; default 20) from the lever's current intensity step up to 1, every run starting from the state reached so far with the earlier levers in place:

$$A(\ell|j)(xi) = \Phi(\, A(j\text{-}1),\ Lj\text{-}1 \oplus (\ell,\ xi)\,)$$

where L ⊕ (ℓ, x) adds lever ℓ at intensity x, or raises an already implemented lever to x. The conditional transition point is the marginal-benefit transition point (Section 12.1) of the outcome gain along this sweep, with s = +1 because G already measures improvement:

$$MB(i) = [\ G(A(\ell|j)(xi{+}1)) - G(A(\ell|j)(xi))\ ] \,/\, \Delta x, \quad i^* = \text{argmin over i of } (MB(i) - MB(i\text{-}1)) \,/\, \Delta x$$

$$\tau\ell|j = xi^*, \quad \text{or 1 if the marginal benefit does not decline before full intensity}$$

Under the default rule the stage implements the lever at uj = τℓ|j, so that the conditional transition point is both the implementation target and the staging point from which further effort is better invested in the next lever; A(j) is the equilibrium at that intensity and Lj = Lj-1 ⊕ (ℓj, uj). Implemented levers stay in place and intensities are never lowered. If the gain at the transition point does not exceed the gain already reached, G(A(j-1)), by more than 10^-9, the lever has no conditional transition point at that stage and is not added there (the branch is pruned). Under the reference rule every stage implements its lever at uj = 1. When scale-up stages are allowed, a lever implemented below 1 may appear once more in a later stage and is then raised to 1.

Because a stage under the default rule proceeds only when the gain at its transition point exceeds the gain already reached, such a stage always raises G. A stage at full intensity (the reference rule, or a scale-up stage) can lower G when the lever worsens the outcomes beyond its transition point; such stages are retained and reported. Version 0.4.2 and earlier stopped a stage at the upper end of the step with the largest positive marginal return instead, so pathway results computed with those versions differ from those of later versions.

The effect of the preceding stages on a lever is its TP shift, δτℓ|j = τℓ|j - τℓ, where τℓ is the lever's transition point on its own from BAU. A negative shift means enabling (the lever's returns diminish sooner in the state reached, so less of it is needed for most of its benefit), a positive shift delaying. The shift is not defined for scale-up stages.

13.4 Lever profiles and the candidate pool

Before pathways are built, every lever is swept from BAU on its own, which is the single-stage case of Section 13.3. A lever is excluded from pathway construction if every outcome's range across the sweep is below the flat-response threshold ε (default 0.015), or if G at its transition point is not positive. Its stand-alone response type applies the classification of Section 12.2 to τℓ (Proportional if τℓ = 1 by the rule above, Early τℓ ≤ 1/3, Late τℓ ≥ 2/3, Middle in between), and its full-intensity effects on the outcomes are classified with the synergy rule of Section 9.2. Levers with trade-offs are retained, since a later stage may compensate for them. For each outcome, the lever profile also reports that outcome's own transition point, computed on the direction-oriented outcome value by the same marginal-benefit rule, and not reported if the outcome has not improved by then.

13.5 Candidate construction

- **Every order (exhaustive).** A depth-first enumeration of the prefix tree of ordered sequences of up to J stages (J ≤ 5 in the interface). Every prefix is itself a candidate pathway, and pathways that share their first j stages share the equilibrium A(j), which is computed once. The number of ordered sequences of J distinct levers drawn from m is m!/(m - J)!, so enumeration is capped at 20,000 candidates; the interface refuses to start an exhaustive run whose count exceeds the cap.
- **Order constraints.** A constraint "ℓa only after ℓb" admits ℓa at a stage only if ℓb has already been implemented. Constraints apply under every construction strategy.
- **Beam search.** At each depth, all extensions of the retained partial pathways are evaluated; the converged ones are ranked by TOG (or by PEI, as chosen) and the best w (default 10) are retained for the next depth.

- **Greedy.** Beam search with w = 1, ranking the extensions by the largest marginal benefit of any step in the sweep of their latest stage.

Under every strategy, each evaluated extension is a candidate pathway. The heuristic strategies do not guarantee that the best pathway is found.

13.6 Pathway metrics

For a pathway with final state A(J), final lever intensities uℓ(J), and stage intensity increments Δuj:

> $\mathrm{TOG} = G(A(J))$

> $\mathrm{CII} = \Sigma j\, c\ell j \cdot \Delta uj = \Sigma\ell\, c\ell \cdot u\ell(J)$

> $\mathrm{PEI} = \mathrm{TOG} / \mathrm{CII} \quad$ (not defined when $\mathrm{CII} = 0$)

> average TP = mean of $\tau\ell j|j$ over the stages with a defined conditional transition point

> SS = number of outcomes with $dk \cdot (Ak(J) - Ak(BAU)) > \varepsilon$

> trade-offs = number of outcomes with $dk \cdot (Ak(J) - Ak(BAU)) < -\varepsilon$

The Early Benefit Index is the area under the benefit accrual curve. Within every stage, the cumulative intensity and the outcome gain are recorded at each sampled step from the stage's start to its target, giving points (Cm, Gm); starting from (0, 0), the area is computed with the trapezoidal rule on the normalized curve:

> $\mathrm{EBI} = \Sigma m\, (\tilde{C}m - \tilde{C}m\text{-}1) \cdot (\tilde{G}m + \tilde{G}m\text{-}1) / 2$

> $\tilde{C}m = Cm / \mathrm{CII}, \quad \tilde{G}m = Gm / \mathrm{TOG}$

EBI is defined only when TOG and CII are both positive. Values above 0.5 indicate that benefits accrue ahead of effort, values below 0.5 delayed benefits. Tracing the curve at the sweep resolution, rather than only at the stage end points, keeps single-stage pathways classifiable: with stage end points alone every single-stage pathway would have EBI = 0.5. The Interaction Gain compares the pathway with the sum of its levers implemented on their own from BAU at the same final intensities, read from the stand-alone sweeps of Section 13.4:

> $\mathrm{IG} = \mathrm{TOG} - \Sigma\ell\, G(\, \Phi(A(BAU), \{(\ell, u\ell(J))\})\,)$

A positive IG indicates mutual reinforcement; a negative IG that the levers' separate effects overlap or counteract. The mean TP shift averages δτ over the stages where it is defined. An outcome with a desired level tk counts as reached when $dk \cdot (Ak(J) - tk) \geq 0$.

PEI is the effort-weighted mean of the stage efficiencies ej = ΔGj / (cℓj · Δuj). Implementing each lever only up to its point of diminishing returns keeps those stage efficiencies high, which is the sense in which PEI generalizes the efficiency logic of a transition point from one lever to a sequence.

13.7 Admissibility, archetypes, and the efficient frontier

Classification is restricted to admissible pathways. With TOG* the largest TOG among converged candidates and s the minimum-gain share (default 0.25), the minimum meaningful gain is

$$\mathrm{Gmin} = \max(\, s \cdot \mathrm{TOG}^{*},\ \varepsilon \cdot |O| \,)$$

A pathway is admissible if every stage converged, TOG* > 0, TOG ≥ Gmin, and it has no trade-offs. Pathways meeting the gain condition but with trade-offs are reported as conditional. Quantiles Qp are computed across the admissible pathways by linear interpolation:

Archetype	Decision rule (admissible pathways)	Ranking indicator
Low-Hanging Fruit	CII ≤ Q25(CII); average TP ≤ 1/3; EBI > 0.5	CII, ascending
High-Impact	TOG ≥ Q80(TOG); SS = |O|	TOG, descending
Efficiency	PEI ≥ Q80(PEI); Q25(CII) < CII < Q75(CII)	PEI, descending
Deep Investment	CII ≥ Q75(CII); average TP > 1/2; EBI < 0.5	TOG, descending

A pathway may meet several rules. Its primary archetype is the one with the highest affinity, where affinity is the percentile rank (the share of admissible pathways below, ties counted half) on the archetype's defining indicator, reversed where lower is better: 1 - PR(CII) for Low-Hanging Fruit, PR(TOG) for High-Impact, PR(PEI) for Efficiency, and 1 - PR(EBI) for Deep Investment. Within an archetype, ties in the ranking indicator are broken by the synergy score, then the EBI (both descending), then the shorter pathway. Sorted by CII (ties by TOG, descending), an admissible pathway is on the efficient frontier if its TOG exceeds that of every pathway requiring less effort. For each archetype, the tool also counts how many admissible pathways meet each condition on its own, so that an empty archetype can be explained.

> **Status of these rules:** The minimum-gain share, the quartile and quintile thresholds, the average-TP bounds 1/3 and 1/2, and the EBI bound 0.5 are conventions chosen for this tool to structure comparison between pathways; they are not values from the literature. Because they are relative to the admissible set of a run, an archetype describes a pathway compared with the alternatives evaluated in that run, and with few admissible pathways the thresholds are coarse (the interface says so below 10). PEI is scale-invariant, which is why efficiency and impact are separate archetypes, and it is unstable for very small CII, which is one reason it is only interpreted above the minimum gain.

13.8 Implementation portfolios and prefix regret

An opening p is the first stage, or the first two stages, of a pathway, identified by lever and intensity. The prefix regret of an opening is the attainable gain it forecloses:

$$R(p) = \mathrm{TOG}^{*} - \max\{\, \mathrm{TOG}(P) : P \text{ converged and } P \text{ begins with } p \,\}$$

The tool reports every one-stage opening and the ten two-stage openings that begin the largest number of top-ranked pathways (the five best of each archetype), with each opening's regret, that number, and its best continuation per archetype. Openings with zero regret that begin many top-ranked pathways are no-regret first moves. Regret is computed over the evaluated candidates, so under the heuristic strategies it is relative to the pathways found.

13.9 Invariance test for path dependence

If the map has a single globally attracting equilibrium for every lock set, pathways that implement the same levers at the same final intensities end in the same state in any order; order then still matters through the order-

dependent intensities (conditional transition points) and the timing of benefits. If the map is multistable, the state inherited from earlier stages can select the basin of attraction, and order decides the end state itself. The invariance test distinguishes these cases. From the five best pathways of each archetype and the pathways on the efficient frontier, up to 12 distinct lever sets with at least two levers are taken; for each, up to 24 orders of its levers are simulated stage by stage from BAU at the pathway's final intensities, and the largest absolute difference in any concept between the end states is recorded. A set is reported as path dependent if all orders converged and the difference exceeds 0.1, and as undecided if not all orders converged.

The tolerance of 0.1 is a convention. A run stops as soon as no concept changes by more than 10^-9 in one iteration, or at the iteration cap; near a transition point the model settles slowly, and under the relative rule a concept without incoming relationships decays towards zero very slowly, so runs that stop at the cap can end a few hundredths from their equilibrium, differently for different orders. Genuinely different equilibria in a saturating map differ by far more: in the bundled example model (Section 14.5), analysed with version 0.3.0, the differences between orders were at most 0.055 for the sets found invariant and 1.6 to 1.7 for the two sets found path dependent. The test can only detect multistability for the starting states and orders it simulates, so it indicates, but cannot prove, the absence of path dependence.

13.10 Simulation settings and convergence

Every stage is placed at a transition point, close to the steepest part of the system's response. In maps with strong feedback such points lie close to the points where the system tips from one state to another, and near them the approach to equilibrium slows markedly, the phenomenon known as critical slowing down (Scheffer et al., 2009). Pathway runs therefore use their own iteration cap, default 1000 instead of the global 100 and never below 1000, and run on until no concept changes by more than 10^-9 per iteration (or the global threshold, if smaller), because the transition point compares neighbouring changes of the gain curve, which magnifies whatever is left of a run that has not quite settled. A run counts as converged when its last change is below the global convergence threshold, and the number of runs that did not reach 10^-9 is reported. A pathway with a stage that does not converge within the cap, which usually indicates a sustained oscillation (Section 3.4) rather than slow settling, is reported but excluded from classification. Without a squashing function, a stage whose run runs away (Section 3.4) is treated as not converged.

13.11 Robustness to simulation settings

Because FCM results depend on the simulation settings as well as on the weights, the whole analysis can be repeated under four alternative settings: steepness λ halved and doubled, the other smooth squash function (sigmoid instead of tanh, or the reverse), and the other update rule (the λ and squash-function alternatives are omitted under the trivalent function, which has no steepness). Without a squashing function the steepness alternatives are likewise omitted, and the hyperbolic tangent is compared instead. The tracked pathways are the five best of each archetype for which that archetype is primary. They are matched across settings by the order of their levers, not by intensities, which follow each setting's own transition points. For each, the tool reports in how many settings it stays admissible, keeps its primary archetype, and stays among the five best of that archetype.

13.12 Assumptions and limitations

- **Separation of time scales.** Each stage is assumed to last long enough for the system to reach equilibrium before the next begins, and implemented levers are assumed to be sustained. A stage is an ordinal implementation phase, not a period of calendar time.
- **Compensatory gain.** TOG is a weighted sum, so a gain in one outcome can offset a loss in another; admissibility therefore excludes trade-offs, and the synergy score is reported alongside TOG.

- **Effort.** CII treats effort as linear in intensity and commensurable across levers, qualified only by the effort weights, which encode relative demands by ranking rather than measured costs.
- **Search.** The pathway space grows factorially; exhaustive enumeration is capped, and the heuristic strategies may miss the best pathway.
- **Relative classification.** Archetypes and their thresholds are relative to the admissible set of a run (Section 13.7), and the invariance test is indicative only (Section 13.9).
- **Semi-quantitative results.** Gains, intensities, and efficiencies are expressed in the normalized units of the map and are meaningful in comparison between pathways, not as predictions of the magnitude or timing of change.

14. Validity, Assumptions, and Limitations

14.1 Assumptions inherited from the FCM formalism itself

- **Weights and initial values are elicited beliefs, not measured parameters.** A relationship weight of +0.5 encodes that stakeholders judged the relationship moderately positive, not a coefficient estimated from data; every result inherits whatever validity the elicitation process itself has.
- **The causal structure is treated as static for the duration of a run.** Relationships and their weights do not change while the model iterates; only concept activations evolve. Structural change is represented by editing the model and re-running, not by anything internal to the update rule.
- **The squash function and λ are non-trivial modelling choices, not cosmetic settings** (Section 3.3). Because they can change which equilibrium, including whether a non-zero one, a given set of weights settles into, they should always be reported alongside any result quoted from the tool. The option of running without a squashing function shows what the weights alone imply, and so how much the bound contributes.
- **Equilibria may depend on the starting state** (Section 3.6). A single baseline run shows one equilibrium; its uniqueness should be checked from several starting vectors before it is reported as the system's equilibrium.

14.2 Assumptions introduced by modules and scope

- **Module boundaries are the modeller's.** Every between/within classification, module statistic, interface network, and modularity value depends on how concepts were assigned to modules. The modularity value of Section 5.7 indicates whether the assignment follows the network's structure, but does not validate it.
- **Subsystem and interface results answer different questions from full-system results** (Section 4.3). They are informative in comparison with the full system and should not be reported in place of it.

14.3 Method-specific limitations

- Influence/Sensitivity scores (Section 6) are one-factor-at-a-time and do not capture concept-concept interaction effects; use a multi-intervention scenario (Section 9) to test those directly.
- Betweenness (Section 5.4) is computed on the unweighted graph and ignores relationship strength; clustering (Section 5.6) ignores weights and directions.
- The multi-scale "importance mainly from" label and the route bottleneck flag rest on thresholds chosen for this tool (Sections 7.3 and 8.4).
- Asynchronous updating (Section 3.5) is inherently order-dependent; results from asynchronous runs are not guaranteed reproducible between runs of an identical model. (Monte Carlo runs are reproducible from their

seed.)

- Transition Point Analysis (Section 12) is direction-agnostic: it counts benefit in the direction an outcome moves, so it identifies where returns start to diminish, not whether the response is desirable. Its transition point compares neighbouring changes of a sampled curve, so it depends on the sweep resolution and on runs that have not fully settled, and it is local: when the marginal benefit dips and later rises again, the transition point marks the dip.
- **Monte Carlo simulation (Section 11)** samples weights and starting values only, independently and uniformly with one spread for all of them; its output is a distribution of equilibria under that assumption, not a probability, and runs that do not settle are included and flagged.
- **Running without a squashing function (Sections 3.3 and 3.4)** shows the unbounded linear dynamics; most maps with reinforcing feedback run away, and the runaway limit of 1000 is a safeguard, so a run stopped at it has no equilibrium.
- Transition Pathway Analysis (Section 13) assumes that each stage settles before the next begins and that implemented levers are sustained; its archetypes are relative to the pathways of a run, its gain measure is compensatory, and its invariance test is indicative only (Section 13.12).
- Feedback-loop enumeration (Sections 5.10 and 8.6) is capped at 300 loops of up to 8 relationships, and route enumeration (Section 8.1) at 20,000 routes of up to 6 relationships; a model beyond these limits reports partial, explicitly flagged figures.
- Very large models (roughly 100+ concepts) remain simulable, but the multi-scale analysis and some interface elements become slow at that scale; this is a software-performance limitation, not a methodological one.

14.4 What this tool does not claim

Two boundaries are worth stating explicitly, since they are easy to blur when reporting FCM results:

- **Not a forecast.** Every scenario, sensitivity, transition-point, transition-pathway, route, and loop result is an implication of the elicited causal structure under the stated assumptions (update rule, squash function, λ, analysis scope): what that specific structure, as drawn, entails, not a prediction of what will actually happen in the real system being modelled. This framing is surfaced in the tool's own interface text for this reason.
- **Not causal discovery.** The tool never infers the causal structure itself from data; the structure, and its division into modules, is always specified by the modeller or elicited from stakeholders beforehand, and every method in this annex operates on that already specified structure. Nothing in this tool performs causal inference in the statistical sense.

14.5 The former example models

From version 0.5.0 the tool ships without example models: it starts with an empty model and a guide to building one. The two examples it offered before are documented here because results reported with earlier versions, including the case study of the transition pathways method, were obtained with them. Their definitions are kept in the source repository (tests/fixtures/example-models.js), where the regression tests use them.

The Water-energy-food nexus example, shipped from version 0.2.1 to 0.4.5, is a teaching and demonstration model. Its concepts, relationships, and weights were chosen to illustrate how the tool handles a modular nexus; they are not derived from data, literature synthesis, or stakeholder elicitation, and results obtained from it say nothing about any real system. Its construction is documented here so that it is not mistaken for evidence:

- **Structure.** 40 concepts in five modules of eight (Water, Energy, Food, Ecosystems & Climate, Governance & Society), each with a written definition; 90 relationships, 50 within modules and 40 between them, with weights

on the tool's 0.25-step scale. The modularity of the module partition (Section 5.7) is about 0.37, so the modules are clearly more densely connected internally than to each other.

- **Policy levers.** Climate and energy policy and agri-environmental measures have no incoming relationships, so their value is set only by a scenario. This follows the common FCM practice of representing policy instruments as exogenous drivers.
- **Adjustments for convergence.** A first version of the model oscillated under the default settings (relative rule, tanh, λ = 1). It was made convergent by removing four relationships (climate warming → public environmental awareness, public environmental awareness → climate and energy policy, public environmental awareness → agri-environmental measures, and population and economic growth → total water demand) and by weakening renewable energy capacity → fossil fuel use from -0.75 to -0.25. The baseline converges in 73 iterations and the three scenarios in 67 to 69, under the default settings. These are modelling choices made for a clean demonstration, of the kind a modeller would otherwise have to justify substantively (see Section 3.4).
- **Behaviour.** Without policy, the baseline settles in a degraded state (high emissions and warming, low food security and water availability). The three scenarios show two distinct patterns of module coupling: holding climate and energy policy at +1 moves every module, while holding agri-environmental measures at +1 changes mainly the Food, Ecosystems & Climate, and Water modules and leaves Energy practically unchanged.
- **Pathways.** With food security, biodiversity, and greenhouse gas emissions (lower is better) as outcomes and six levers (climate and energy policy, agri-environmental measures, water allocation policy, research and innovation, energy efficiency, and wastewater treatment capacity), up to three stages, and default settings, the analysis evaluates 75 pathways, of which 47 are admissible and 27 have a stage that does not converge within 500 iterations. Wastewater treatment capacity is excluded as flat. Because one policy lever already moves the whole system, many pathways reach similar gains at very little effort: 9 are Low-Hanging Fruit, 10 High-Impact, 1 Deep Investment, and none Efficiency, since every highly efficient pathway lies in the lowest effort quartile. Two of the nine lever sets tested are path dependent (Section 13.9). These results, computed with version 0.3.0 (later versions define the transition point differently and give other numbers), illustrate the method on a saturating map and, like the model itself, say nothing about any real system.
- **Uncertainty (version 0.4.0).** With every weight varied by ±0.1 (500 runs, seed 2026), only 135 of the baseline's runs settle within 100 iterations, and 236 within 500, although the unperturbed baseline settles in 73. The model lies close to an oscillation boundary, which follows from how it was made convergent (see the adjustments above): it is a demonstration model, not a robust one, and Monte Carlo shows that plainly.

A second example, added in version 0.4.0, is different in kind: it is the illustrative case study of the pathway method, and its results can be checked against a published set of tables.

- **Structure.** The peat meadow nexus (food, climate, and biodiversity in Dutch peat meadow polders) has 20 concepts in five modules (Interventions, Water & soil, Farming system, Nexus outcomes, Socio-economic), 50 relationships, and seven scenarios (the baseline, each of the five levers at full intensity, and a bundle of all five). Weights, starting values, definitions, and positions are those of Annex 1 of "From transition points to transition pathways". Its tables were computed with the transition point of version 0.4.2 and earlier, so transition points and pathways from later versions differ from them. Every lever responds slowly near its transition point, so the example loads with a limit of 300 iterations.
- **Reproduction.** Under the default settings with 300 iterations, the baseline converges in 16 iterations and reproduces the baseline table of the case study (largest difference 0.005), the five lever profiles reproduce its table of achievable ranges and response types, and the pathway analysis reproduces its pathway table (188 pathways, 14 admissible, with the archetypes and the total outcome gain, cumulative intervention intensity, pathway efficiency, and interaction gain of the case study to three decimals).
- **Without squashing.** The baseline runs away and is stopped at iteration 13, with the largest activation about 1,700: the two reinforcing loops of the model (the subsidence spiral and the intensification loop) amplify

themselves on every pass, which the squashing function hides by saturating the concepts.

- **Uncertainty.** Monte Carlo on the baseline (1,000 runs, seed 2026). With weights varied by ±0.1 every run settles and the three outcomes hardly move (their standard deviations are below 0.01, because the baseline is almost saturated). With ±0.25, 915 of 1,000 runs settle, the standard deviation of farm income is 0.49, and the 95th percentile of meadow bird biodiversity, -0.99 nominally, is +0.55: some runs end in a different state. The baseline's near-total loss of biodiversity is therefore robust to small changes in the weights but not to large ones.

15. Summary Table

Method	Core procedure	Basis
Update rule	A(t+1) = f(A(t) + W·A(t)) [relative] or f(W·A(t)) [absolute]	Kosko (1986); relative form widely used in later FCM tools
Squashing functions	tanh(λx) / 2(1+e^-λx)^-1 - 1 (= tanh(λx/2)) / ±1 step / identity (no squashing)	Standard FCM squash functions; gain note verified for this tool; the identity option is this tool's own
Custom initial state	Baseline run from a chosen A(0); multi-start uniqueness check	Standard dynamical-systems practice; implementation is this tool's own
Modules and analysis scope	Normalized categories; induced subgraph or between-module edge set	This tool's own operationalization (Section 4)
Folded (sub-model) display	Contraction of folded modules; aggregate links with count and net weight; display only	Standard graph contraction; this tool's own display (Section 4.5)
Integrity checks on opening	Deterministic repair or removal of malformed entries, listed in a notice	This tool's own implementation (Section 4.6)
Automatic layout (smart Arrange)	Louvain groups with the Leiden connectivity guarantee; group and coarse force-directed layouts; rotation; local search on a weighted readability objective; display only	Blondel et al. (2008); Traag et al. (2019); Fruchterman & Reingold (1991); objective weights and modes are this tool's own (Section 4.7)
Degree centrality / role	Weighted in/out-degree; 75th-percentile and 1.2× thresholds	Standard graph-theoretic centrality; thresholds are this tool's own
Driver / receiver score	Outdegree or indegree ÷ centrality	Normalization of degree; this tool's own reporting
Betweenness	Shortest-path brokerage, unweighted, normalized by (n-1)(n-2)	Freeman (1977); Brandes (2001) algorithm
Density / connectivity / centralization	Standard network descriptive statistics	Standard graph theory; centralization after Freeman (1978)
Clustering coefficient	Mean local clustering on the undirected projection	Watts & Strogatz (1998)

Method	Core procedure	Basis
Modularity	Q of the modeller's module partition, |w|-weighted	Newman & Girvan (2004); applied to a given, not optimized, partition
Complexity index	Receivers ÷ Drivers	Özesmi & Özesmi (2004)
Feedback loop sign	Even/odd count of negative edges around a cycle	Maruyama (1963); parity rule is standard signed-graph convention
Influence / Sensitivity scores	OAT perturbation to +1 from an all-zero state	General OAT design (Saltelli et al., 2007); zero-state procedure is this tool's own
Multi-scale analysis	Metrics recomputed per scale; competition ranks; percentile rule	Original analysis developed for this tool
Influence routes	Bounded simple-path enumeration; product strength; bridge share; bottleneck ≥ 50%	Path-based reasoning after Kosko (1986); measures and thresholds are this tool's own
Feedback loops by module	Within/between classification of enumerated cycles	This tool's own breakdown of the Section 5.10 loops
Scenario drivers / locks	Fix a concept at an extreme for the whole run	Mental Modeler scenario convention (Gray et al., 2013)
Effect size / synergy / trade-off	Scenario-minus-baseline equilibrium comparison	This tool's own operational definitions
Sensitivity Analysis (tab)	Range sweep + secant elasticity + tornado ranking	Standard local sensitivity-analysis / tornado-diagram practice
Monte Carlo simulation	Uniform random perturbation of unlocked weights (and optionally starting values), R simulations, mean, median, standard deviation, and percentiles of the final states	Monte Carlo method (Metropolis & Ulam, 1949; Saltelli et al., 2007); the sampling design is this tool's own
Transition Point Analysis	Sweep; transition point where the marginal benefit declines most steeply + response-type classification	Original method developed for this tool
Transition Pathway Analysis	Sequential sweeps from the state reached; conditional transition points on the signed outcome gain; seven metrics; relative archetype rules; efficient frontier; prefix regret; invariance and robustness tests	Original method developed for this tool, motivated by the pathways, path-dependence, and leverage-points literatures (Section 13)

16. References

Abson, D. J., Fischer, J., Leventon, J., Newig, J., Schomerus, T., Vilsmaier, U., von Wehrden, H., Abernethy, P., Ives, C. D., Jager, N. W., & Lang, D. J. (2017). Leverage points for sustainability transformation. Ambio, 46(1), 30–39. https://doi.org/10.1007/s13280-016-0800-y

Arthur, W. B. (1989). Competing technologies, increasing returns, and lock-in by historical events. The Economic Journal, 99(394), 116–131. https://doi.org/10.2307/2234208

Axelrod, R. (Ed.). (1976). Structure of Decision: The Cognitive Maps of Political Elites. Princeton University Press.

Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. Journal of Statistical Mechanics: Theory and Experiment, 2008(10), P10008. https://doi.org/10.1088/1742-5468/2008/10/P10008

Bonacich, P. (1972). Factoring and weighting approaches to status scores and clique identification. The Journal of Mathematical Sociology, 2(1), 113-120. https://doi.org/10.1080/0022250X.1972.9989806

Bostock, M., Ogievetsky, V., & Heer, J. (2011). D3: Data-driven documents. IEEE Transactions on Visualization and Computer Graphics, 17(12), 2301-2309. https://doi.org/10.1109/TVCG.2011.185

Brandes, U. (2001). A faster algorithm for betweenness centrality. The Journal of Mathematical Sociology, 25(2), 163–177. https://doi.org/10.1080/0022250X.2001.9990249

Fischer, J., & Riechers, M. (2019). A leverage points perspective on sustainability. People and Nature, 1(1), 115–120. https://doi.org/10.1002/pan3.13

Freeman, L. C. (1977). A set of measures of centrality based on betweenness. Sociometry, 40(1), 35–41. https://doi.org/10.2307/3033543

Freeman, L. C. (1978). Centrality in social networks: Conceptual clarification. Social Networks, 1(3), 215–239. https://doi.org/10.1016/0378-8733(78)90021-7

Fruchterman, T. M. J., & Reingold, E. M. (1991). Graph drawing by force-directed placement. Software: Practice and Experience, 21(11), 1129-1164. https://doi.org/10.1002/spe.4380211102

Gray, S. A., Gray, S., Cox, L. J., & Henly-Shepard, S. (2013). Mental Modeler: A fuzzy-logic cognitive mapping modeling tool for adaptive environmental management. In 2013 46th Hawaii International Conference on System Sciences (pp. 965–973). IEEE. https://doi.org/10.1109/HICSS.2013.399

Haasnoot, M., Kwakkel, J. H., Walker, W. E., & ter Maat, J. (2013). Dynamic adaptive policy pathways: A method for crafting robust decisions for a deeply uncertain world. Global Environmental Change, 23(2), 485–498. https://doi.org/10.1016/j.gloenvcha.2012.12.006

Kosko, B. (1986). Fuzzy cognitive maps. International Journal of Man-Machine Studies, 24(1), 65–75. https://doi.org/10.1016/S0020-7373(86)80040-2

Maruyama, M. (1963). The second cybernetics: Deviation-amplifying mutual causal processes. American Scientist, 51(2), 164–179.

Metropolis, N., & Ulam, S. (1949). The Monte Carlo method. Journal of the American Statistical Association, 44(247), 335–341. https://doi.org/10.1080/01621459.1949.10483310

Newman, M. E. J., & Girvan, M. (2004). Finding and evaluating community structure in networks. Physical Review E, 69(2), 026113. https://doi.org/10.1103/PhysRevE.69.026113

Özesmi, U., & Özesmi, S. L. (2004). Ecological models based on people's knowledge: a multi-step fuzzy cognitive mapping approach. Ecological Modelling, 176(1–2), 43–64. https://doi.org/10.1016/j.ecolmodel.2003.10.027

Papageorgiou, E. I., & Salmeron, J. L. (2013). A review of fuzzy cognitive maps research during the last decade. IEEE Transactions on Fuzzy Systems, 21(1), 66–79. https://doi.org/10.1109/TFUZZ.2012.2201727

Purchase, H. (1997). Which aesthetic has the greatest effect on human understanding? In G. DiBattista (Ed.), Graph Drawing (GD 1997), Lecture Notes in Computer Science 1353 (pp. 248-261). Springer. https://doi.org/10.1007/3-

540-63938-1_67

Saltelli, A., Ratto, M., Andres, T., Campolongo, F., Cariboni, J., Gatelli, D., Saisana, M., & Tarantola, S. (2007). Global Sensitivity Analysis: The Primer. Wiley. https://doi.org/10.1002/9780470725184

Scheffer, M., Bascompte, J., Brock, W. A., Brovkin, V., Carpenter, S. R., Dakos, V., Held, H., van Nes, E. H., Rietkerk, M., & Sugihara, G. (2009). Early-warning signals for critical transitions. Nature, 461(7260), 53–59. https://doi.org/10.1038/nature08227

Tarjan, R. (1972). Depth-first search and linear graph algorithms. SIAM Journal on Computing, 1(2), 146-160. https://doi.org/10.1137/0201010

Traag, V. A., Waltman, L., & van Eck, N. J. (2019). From Louvain to Leiden: Guaranteeing well-connected communities. Scientific Reports, 9, 5233. https://doi.org/10.1038/s41598-019-41695-z

Walker, B., Holling, C. S., Carpenter, S. R., & Kinzig, A. P. (2004). Resilience, adaptability and transformability in social-ecological systems. Ecology and Society, 9(2), Article 5. https://doi.org/10.5751/ES-00650-090205

Watts, D. J., & Strogatz, S. H. (1998). Collective dynamics of 'small-world' networks. Nature, 393(6684), 440–442. https://doi.org/10.1038/30918

The Spaghetti Engine - Methodological Annex, version 0.5.3

To be read alongside The Spaghetti Engine - User Manual (version 0.5.3).